Day 65: The LSTM Backward Pass From Scratch, a Gradient Check, and a First Training Run

MD Mutasim Billah | 52-Week Data Science to ML/AI Roadmap | Week 10, Day 3 | Complete study guide

Watch: LSTM Backprop Through Time Formula Derivation (Henry Harvin) | CS231n Winter 2016: Lecture 10: Recurrent Neural Networks, Image Captioning, LSTM (Andrej Karpathy)

Script: `day65_lstm_backward_pass.py` · every number in this guide is copied by program from `day65_run_output.txt`, the real output of that script.

Day 64 cached every gate of the forward pass. Today that cache is walked backwards to get gradients, the gradients are checked against numerical estimates, and, for the first time in this week, an LSTM is **trained**. The task is Day 59's: the label is the sign of the first number in a sequence of random numbers. The vanilla RNN, with the same data and settings, reached 0.955 at 20 steps and 0.525 at 40 on Day 60. The LSTM's result depends almost entirely on one number set before training starts: the forget gate's bias. At bias 0 it failed every one of 18 runs, including at 20 steps; at bias 2 it learned all 12 runs at 20 and 40 steps and 4 of 6 at 100.

0. Where today fits

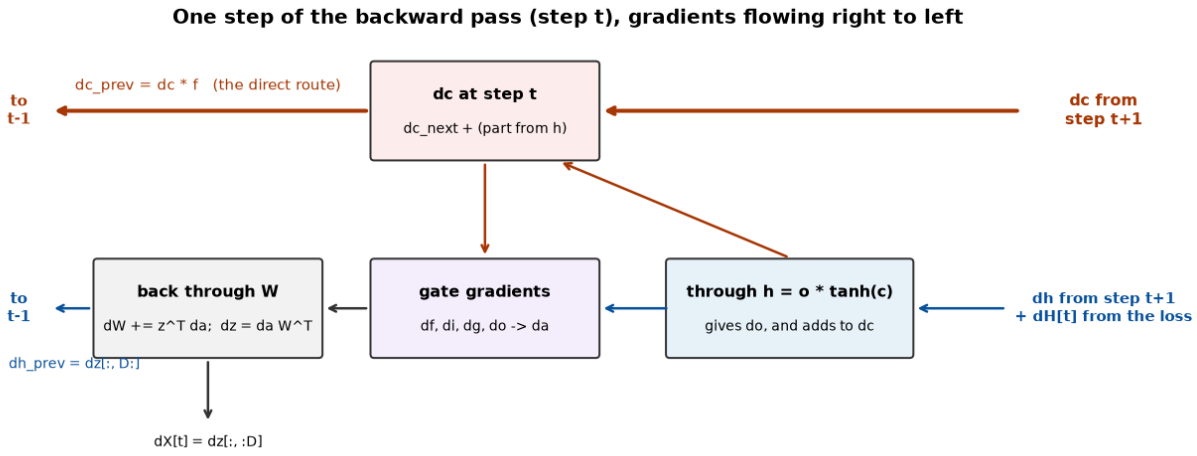
DEFINITION — start here, no prior knowledge assumed

The **backward pass** works out, for every weight, how much a small change in it would change the loss: the **gradient**. It runs the forward pass's steps in reverse, applying the chain rule at each one. In an LSTM two gradients flow back in time side by side: **dh** for the hidden state and **dc** for the cell state.

REAL-WORLD SCENARIO

A cake comes out of the oven too dry. To find out why, you go back through the recipe in reverse: the baking time, then the oven temperature, then how much butter went in, asking at each step how much that step contributed to the dryness. The written record of what you did at each step (the cache) is what makes this possible without baking the cake again.

SYSTEM DIAGRAM



One step of the backward pass. dc (orange) arrives from the next step, collects a share from h, feeds the gate gradients, and leaves multiplied by the forget gate. dh (blue) goes through the gates and W.

How to read each section

Step	What it gives you
DEFINITION	What the idea is, in plain words. Start here.
REAL-WORLD SCENARIO	An everyday situation where the same idea shows up.
MATHEMATICS	The formula, when the section introduces one.
SPECIFICATION	The exact setup of the experiment.
Why this test	The question the experiment answers.
IMPLEMENTATION	The real code from the script.
ACTUAL OUTPUT	What the script really printed, copied unchanged.
What it means	How to read the numbers, and what they do and do not show.
Why choose this? / Why NOT this?	When the idea is the right tool, and what it costs.

1. The backward pass on its own

DEFINITION — start here, no prior knowledge assumed

lstm_backward receives the gradient of the loss with respect to the outputs (every hidden state, the last hidden state, the last cell state) and the cache. Going from the last step to the first, it turns dh and dc into gradients for the four gates, then into gradients for W, b, the inputs X and, at the end, the starting states h0 and c0.

A **gradient check** compares those gradients with ones measured by nudging each number up and down and watching the loss. If they agree for every entry, the backward pass is right.

REAL-WORLD SCENARIO

An accountant's audit. The final figure is wrong by some amount. The auditor goes back through the ledger one transaction at a time, and for each one writes down how much it contributed. Spot-checking a few transactions by recalculating them from scratch (the gradient check) is how you know the audit itself was done correctly.

MATHEMATICS

$h = o \cdot \tanh(c)$: $do = dh \cdot \tanh(c)$, $dc \mathrel{+}= dh \cdot o \cdot (1 - \tanh(c)^2)$

$c = f \cdot c_{prev} + i \cdot g$: $df = dc \cdot c_{prev}$, $di = dc \cdot g$, $dg = dc \cdot i$, $\mathbf{dc_{prev} = dc \cdot f}$

through each squashing function: $\text{sigmoid}' = s(1 - s)$, $\tanh' = 1 - \tanh^2 \rightarrow da = [df', di', dg', do']$

$dW \mathrel{+}= [x_t, h_{prev}]^T da$, $db \mathrel{+}= \Sigma\, da$, $[dx_t, dh_{prev}] = da\, W^T$

padding step ($m_t = 0$): dh and dc pass through unchanged, nothing is added to dW

SPECIFICATION

Data	4 sequences of lengths 5, 2, 4, 1 (padded to 5), 2 inputs, 3 hidden units, random weights and biases, random non-zero h0 and c0.
Loss	L = sum(R1 · every h_t) + sum(R2 · last h) + sum(R3 · last c) with fixed random R1, R2, R3, so gradient enters at all three places the function accepts it.
Checked	Every entry of W, b, X, h0 and c0: 136 numbers.
Pass rule	Day 62's: $\lvert\text{numeric} - \text{analytic}\rvert \le 1e\text{-}10 + 1e\text{-}6 \cdot \lvert\text{numeric}\rvert$ for every entry.

Why this test

The LSTM backward pass has more places to go wrong than any so far: four gates, two state streams, two squashing functions, padding. A loss that only read the last hidden state would leave the dH path, the dc entry point and the starting-state gradients untested. This loss uses all of them, so every line of the function is exercised.

IMPLEMENTATION

```
def lstm_backward(dH, dh_last, dc_last, cache):
    X, h0, c0, W, mask = cache["X"], cache["h0"], cache["c0"], cache["W"], cache["mask"]
    Hs, Cs, F, I, G, O = (cache[k] for k in ["H", "C", "F", "I", "G", "O"])
    m, T, D = X.shape
    dW, db, dX = np.zeros_like(W), np.zeros(W.shape[1]), np.zeros_like(X)
    dh, dc = dh_last.copy(), dc_last.copy()
    for t in reversed(range(T)):
        dh = dh + dH[:, t]
        m_t = mask[:, t:t + 1]
        h_prev = Hs[:, t - 1] if t > 0 else h0
        c_prev = Cs[:, t - 1] if t > 0 else c0
        f, i, g, o = F[:, t], I[:, t], G[:, t], O[:, t]
        tanh_c = np.tanh(Cs[:, t])
        dh_real, dc_real = dh * m_t, dc * m_t
        do = dh_real * tanh_c
        dc_new = dc_real + dh_real * o * (1.0 - tanh_c ** 2)
        da = np.concatenate([dc_new * c_prev * f * (1.0 - f),
                             dc_new * g * i * (1.0 - i),
                             dc_new * i * (1.0 - g ** 2),
                             do * o * (1.0 - o)], axis=1)
        z = np.concatenate([X[:, t], h_prev], axis=1)
        dW += z.T @ da
        db += da.sum(axis=0)
        dz = da @ W.T
        dX[:, t] = dz[:, :D]
        dh = dz[:, D:] + dh * (1.0 - m_t)
        dc = dc_new * f + dc * (1.0 - m_t)
    return dW, db, dX, dh, dc
```

The subtle part: the gate gradients use `dc_new`, which already includes the share coming through h = o · tanh(c). Using the incoming dc alone would leave out everything the cell state contributed to this step's output.

ACTUAL OUTPUT

```
  array  entries  max relative error  max |difference|    pass
      W       60            4.88e-09          1.60e-10    True
      b       12            2.25e-10          4.76e-10    True
      X       40            1.72e-10          6.74e-11    True
     h0       12            3.07e-10          3.81e-11    True
     c0       12            1.46e-09          3.31e-11    True
every entry passes: True;  gradient at padded inputs exactly zero: True
Actual output from this lesson's run.
```

What it means

Every entry of every array passes: the largest relative error is 4.88e-09 (in W). The gradient reaching padded inputs is exactly zero, as it should be: those inputs never affected anything. The gradients for h0 and c0 also pass, which matters for Day 69, where one network's final states become another's starting states.

WHY CHOOSE THIS?

Writing the backward pass by hand shows exactly where every gradient comes from, which is what makes the forget-gate result in Section 3 visible at all.

WHY NOT THIS? (TRADEOFFS)

It is long and easy to get subtly wrong; a framework's automatic differentiation would produce the same numbers with no code. Without the gradient check, a hand-written version should not be trusted.

2. The same check through the classifier that is trained

DEFINITION — start here, no prior knowledge assumed

A gradient check is only as good as the loss it uses. This one runs through exactly the code that training uses: the LSTM, the output layer reading the last real hidden state, softmax, and the mean cross-entropy over the batch.

REAL-WORLD SCENARIO

Testing each part of a car separately is necessary but not enough; the car also goes on a test drive with everything bolted together, because that is how it will be used. Day 44 of this series found a gradient that was exactly 2× wrong only because a separate test used a different loss.

SPECIFICATION

Network	LSTMClassifier: 2 inputs, 3 hidden units, 2 classes, forget bias 1 plus random biases.
Data	The same four padded sequences; labels 0, 1, 1, 0.
Checked	Every entry of W, b, Why and by (80 numbers), with Day 62's pass rule.

Why this test

Section 1 tested lstm_backward with an artificial loss. Training will call it from the classifier, with the averaging over the batch done there. Checking the real path is the rule this series learned the hard way.

IMPLEMENTATION

```
def gradients(self, y):
    m = len(y)
    dlogits = self.probs.copy()
    dlogits[np.arange(m), y] -= 1.0
    dlogits /= m
    grads = dict(Why=self.h_last.T @ dlogits, by=dlogits.sum(axis=0))
    dh_last = dlogits @ self.Why.T
    grads["W"], grads["b"], self.dX, _, _ = lstm_backward(np.zeros_like(self.Hs), dh_last,
        np.zeros_like(dh_last), self.cache)
    return grads
```

ACTUAL OUTPUT

```
  array  entries  max relative error  max |difference|    pass
      W       60            1.99e-07          5.56e-11    True
      b       12            1.84e-09          1.25e-10    True
    Why        6            9.38e-11          6.62e-12    True
     by        2            2.83e-12          1.03e-12    True
every entry passes: True
Actual output from this lesson's run.
```

What it means

All four arrays pass (largest relative error 1.99e-07). The classifier's gradients are correct, so any failure to learn in Section 5 is not a bug in the gradients.

WHY CHOOSE THIS?

Checking the exact training path catches mistakes in how the pieces are joined, not just in the pieces.

WHY NOT THIS? (TRADEOFFS)

It is slow (two forward passes per number checked), so it is only practical on a tiny network.

3. The direct route, as a gradient

DEFINITION — start here, no prior knowledge assumed

Day 63 showed that a change to the cell state travels forward along the cell-state line, multiplied at each step by the forget gate. Going backward it is the same line in reverse: $\mathbf{dc_{prev} = dc \cdot f}$. If the gates cannot see h, that line is the only route from c_T back to c_0, so the gradient at c_0 must be the gradient at c_T times the product of all the forget gates, exactly.

REAL-WORLD SCENARIO

Day 63's written note passed down a line of 30 people, each tearing a strip off. Going backward is like asking how much of the final note came from the first person's handwriting: exactly the fraction that every tear left behind, multiplied together. If the people also whispered to each other (the routes through h), the answer would include those too.

MATHEMATICS

gates independent of h: $dL/dc_0 = dL/dc_T \cdot f_1 \cdot f_2 \cdot \ldots \cdot f_T$ *exactly*

normal LSTM: dL/dc_0 *= (direct route) + (routes through h and the gates)*

SPECIFICATION

Setup	16 units, 200 sequences of 30 N(0,1) inputs, forget bias 2, random weights (seed 3). A random gradient is placed on c_T and nothing else.
Two versions	The rows of W that multiply h set to zero (no indirect route), and the normal LSTM.
Compared	The backward pass's dL/dc_0 against dL/dc_T times the product of the forget gates.

Why this test

It ties today's code to Day 63's argument with a check that has an exact answer, and it shows how much of the gradient in a normal LSTM travels by the direct route and how much by the others.

IMPLEMENTATION

```
for label, cut in [("gates cut off from h", True), ("normal LSTM", False)]:
    Wc = W.copy()
    if cut:
        Wc[1:, :] = 0.0
    _, _, cache = lstm_forward(X, zeros, zeros, Wc, b)
    _, _, _, _, dc0 = lstm_backward(np.zeros((m, T, H)), np.zeros((m, H)), dc_T, cache)
    direct = dc_T * np.prod(cache["F"], axis=1)
```

ACTUAL OUTPUT

```
gates cut off from h: ||dL/dc_0|| = 1.184025e+00,  direct route alone = 1.184025e+00,  max
    |difference| = 6.94e-17
normal LSTM: ||dL/dc_0|| = 6.125373e+01,  direct route alone = 1.313349e+00,  max |difference| =
    4.75e+00
Actual output from this lesson's run.
```

What it means

With no indirect route, the backward pass gives exactly the product of forget gates (difference 6.94e-17): the line `dc = dc_new * f` is the direct route.

In the normal LSTM the total is 6.125373e+01 while the direct route carries only 1.313349e+00. At forget bias 2, for this untrained network, most of the gradient reaching c_0 comes through h and the gates, and it has grown on the way. That matches Day 63, where the indirect route amplified a nudge at forget bias 4: the direct route is the safe, bounded one; the others can grow.

WHY CHOOSE THIS?

The direct route gives gradient a path whose strength is just the product of forget gates, never above 1 and directly controlled by them.

WHY NOT THIS? (TRADEOFFS)

It is not the only path. The indirect routes can dominate and grow, which is why clipping is kept for the training below.

4. How much gradient reaches the first input, before training

DEFINITION — start here, no prior knowledge assumed

In the sign task only the first input x_0 decides the label. Learning can only start if some of the gradient reaches x_0. The ratio $\mathbf{||dL/dx_0|| / ||dL/dx_{T-1}||}$ measures how much reaches the first input compared with the last, for an untrained network.

REAL-WORLD SCENARIO

A teacher grading a long essay writes comments on every page, but the comments are meant to improve the opening paragraph. If by the time they reach page 1 they have faded to a whisper, the student never learns what was wrong with the opening, however many essays they write.

SPECIFICATION

Setup	LSTMClassifier, 16 units, seed 7, untrained. Day 59's 400 test sequences at lengths 20, 40, 100.
Varied	Forget bias 0, 1, 2.
Measured	$\|\|dL/dx_0\|\| / \|\|dL/dx_{T-1}\|\|$, from the backward pass (no measurement floor).
Note	Day 59 measured a related ratio for the RNN on hidden states, not inputs; only the orders of magnitude are comparable.

Why this test

Day 63 measured how a nudge travels forward. This is the version that matters for learning, measured exactly, and it predicts Section 5 before any training is done.

IMPLEMENTATION

```
net = LSTMClassifier(1, 16, 2, seed=7, forget_bias=fb)
net.forward(X[:, :, None]); net.gradients(y)
norms = np.linalg.norm(net.dX[:, :, 0], axis=0)
table[(T, fb)] = norms[0] / norms[-1]
```

ACTUAL OUTPUT

```
    T  forget bias 0.0  forget bias 1.0  forget bias 2.0
   20        3.279e-04        5.715e-02        6.973e-01
   40        4.857e-07        1.763e-02        1.524e+00
```

```
  100        3.298e-15        1.189e-03        9.932e+00
Actual output from this lesson's run.
```

What it means

Forget bias 0: 3.279e-04 at 20 steps, 4.857e-07 at 40 and 3.298e-15 at 100. For comparison, Day 59's RNN passed back 1.79e-02 at 20 steps (on hidden states) and learned with clipping. At the default bias the LSTM gives its first input *less* gradient than that RNN.

Forget bias 1: 5.715e-02, 1.763e-02, 1.189e-03. **Forget bias 2:** 6.973e-01, 1.524e+00, 9.932e+00. Raising the bias by 2 changes the gradient reaching x_0 at 100 steps by about fifteen orders of magnitude. Values above 1 mean the gradient grows on the way back (the indirect routes again).

WHY CHOOSE THIS?

One backward pass at initialization predicts whether a long-range task can be learned at all, at the cost of a few milliseconds.

WHY NOT THIS? (TRADEOFFS)

It says whether learning can start, not whether it will succeed; Section 5 is needed for that.

5. The first LSTM training runs

DEFINITION — start here, no prior knowledge assumed

Training repeats: run a batch forward, compute the loss and gradients, clip them (Day 60), and move every weight a small step against its gradient. The **learning rate** sets the step size. A run counts as having learned the task here if its final test accuracy is at least 0.9 (chance is 0.5).

REAL-WORLD SCENARIO

Teaching someone to remember the first word of a long sentence and ignore the rest. If, at the start, they have no way to hold on to that first word (forget bias 0: the memory halves at every word), no amount of practice helps. If they start with the habit of keeping things (forget bias 2), practice turns that habit into the skill.

SPECIFICATION

Task	Day 59's sign task, Day 59's data: 600 training and 400 test sequences, all inputs N(0,1), label = sign of x_0. Lengths 20, 40, 100.
Network	LSTMClassifier, 16 units, reading the last hidden state.
Training	Batch 30, 150 epochs, clipping at global norm 1.0 (Day 60's setup), learning rate 0.05 (Day 60's) and 0.2.
Varied	Forget bias 0, 1, 2; seeds 7, 8, 9 (initial weights and batch order). 54 runs.
Tracked	Test accuracy every 10 epochs; reported as peak / final for each seed.
Reference	Day 60, same data, lr 0.05, clipped at 1.0, vanilla RNN: final 0.955 at 20 steps, 0.525 at 40 (and no clipping threshold held at 40).

Why this test

Days 63 and 64 argued what the forget gate should do. Today, for the first time, the LSTM can be trained, so the argument can be tested on the task the vanilla RNN could not learn. Three seeds per setting, because single runs at these lengths turned out to vary a lot.

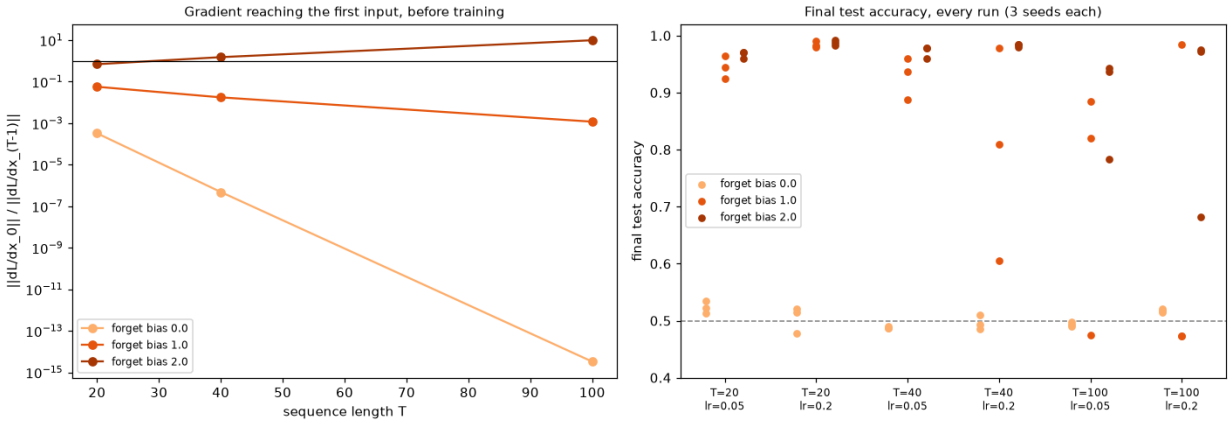
IMPLEMENTATION

```
def train_step(self, X, y):
    self.forward(X)
    grads, norm, _ = clip_by_global_norm(self.gradients(y), self.clip)
```

```
    for name, g in grads.items():
        getattr(self, name)[...] -= self.lr * g
# inside demo_training(): one run
net = LSTMClassifier(1, 16, 2, seed=seed, forget_bias=fb, lr=lr, clip=1.0)
rng = np.random.RandomState(seed)
curve = []
for epoch in range(150):
    order = rng.permutation(600)
    for start in range(0, 600, 30):
        idx = order[start:start + 30]
        net.train_step(X_tr[idx], y_tr[idx])
    if epoch % 10 == 0 or epoch == 149:
        curve.append(net.accuracy(X_te, y_te))
```

ACTUAL OUTPUT

```
   T  f-bias    lr   peak / final test accuracy, seeds 7, 8, 9    final >= 0.9
  20     0.0  0.05   0.565/0.522  0.542/0.535  0.515/0.512              0 of 3
  20     0.0   0.2   0.532/0.515  0.532/0.520  0.530/0.477              0 of 3
  20     1.0  0.05   0.965/0.965  0.945/0.945  0.948/0.925              3 of 3
  20     1.0   0.2   0.980/0.980  0.985/0.983  0.990/0.990              3 of 3
  20     2.0  0.05   0.970/0.970  0.960/0.960  0.970/0.970              3 of 3
  20     2.0   0.2   0.988/0.988  0.985/0.983  0.995/0.993              3 of 3
  40     0.0  0.05   0.492/0.490  0.535/0.487  0.507/0.487              0 of 3
  40     0.0   0.2   0.532/0.510  0.515/0.492  0.510/0.485              0 of 3
  40     1.0  0.05   0.950/0.938  0.905/0.887  0.975/0.960              2 of 3
  40     1.0   0.2   0.810/0.810  0.760/0.605  0.985/0.978              1 of 3
  40     2.0  0.05   0.978/0.978  0.978/0.978  0.980/0.960              3 of 3
  40     2.0   0.2   0.990/0.985  0.990/0.985  0.985/0.980              3 of 3
 100     0.0  0.05   0.542/0.492  0.525/0.490  0.525/0.497              0 of 3
 100     0.0   0.2   0.540/0.520  0.530/0.517  0.525/0.515              0 of 3
 100     1.0  0.05   0.535/0.475  0.892/0.885  0.925/0.820              0 of 3
 100     1.0   0.2   0.527/0.472  0.988/0.985  0.545/0.472              1 of 3
 100     2.0  0.05   0.785/0.782  0.963/0.943  0.963/0.938              2 of 3
 100     2.0   0.2   0.682/0.682  0.985/0.975  0.983/0.973              2 of 3
Actual output from this lesson's run.
```



Left: gradient reaching the first input before training (Section 4). Right: final test accuracy of all 54 runs; each dot is one seed. Dashed line: chance.

What it means

Forget bias 0 never learns. All 18 runs end near chance, at every length and both learning rates; the best final accuracy is 0.535. That includes 20 steps, where Day 60's vanilla RNN reached 0.955 with the same setup. Section 4 predicted it: 3.3e-04 of the gradient reaches x_0 at the start. An LSTM at its plain default bias is worse than a clipped RNN on this task.

Forget bias 2 learns almost everywhere. All 12 runs at 20 and 40 steps end at or above 0.9 (at 40 steps, between 0.960 and 0.985), where Day 60's RNN ended at 0.525. At 100 steps 4 of 6 runs learn; the two that do not end at 0.782 and 0.682.

Forget bias 1 sits in between: every run learns at 20 steps, 3 of 6 at 40 and 1 of 6 at 100.

Results vary between seeds. At 40 steps, forget bias 1 and learning rate 0.2, the three seeds end at 0.810, 0.605 and 0.978. While building this lesson, an earlier version that applied clipping with the same arithmetic in a different order gave slightly different numbers for some of these runs: small differences grow over 3,000 training steps. Read the table by counting how many seeds learn, not by any single number.

What this does not show. It is one task, chosen because it needs memory of the first input and nothing else. The RNN's numbers are Day 60's, not re-tuned today. A fair comparison of trained RNN, LSTM and GRU on real data is Day 67.

WHY CHOOSE THIS?

With its forget gate started open, the LSTM learns a dependency 40 and often 100 steps long that the vanilla RNN could not learn at 40, with the same data, optimizer and clipping.

WHY NOT THIS? (TRADEOFFS)

The advantage depends on a setting the textbook equations do not mention: at forget bias 0 the LSTM did worse than the RNN. Each step also costs several times more (Days 63 and 64), and long runs still vary from seed to seed.

Interview preparation — technical questions

Q: Write the backward step for the LSTM cell state.

A: With h = o * tanh(c): dc += dh * o * (1 - tanh(c)^2). Then df = dc * c_prev, di = dc * g, dg = dc * i, and the gradient passed to the previous cell state is dc * f. Each gate gradient is then multiplied by its activation's slope before going through W.

Q: Which arrays does the LSTM backward pass need from the forward pass?

A: The inputs, h0 and c0, every hidden and cell state, and the four gate values at every step. tanh(c) can be recomputed from c.

Q: How do you gradient-check a recurrent layer properly?

A: Use a loss that sends gradient into every output the layer exposes (all hidden states, the last hidden and cell states), include padding, check inputs and starting states as well as weights, and then check again through the exact training pipeline.

Q: What should you initialize the LSTM forget bias to?

A: A positive value such as 1 or 2. Here forget bias 0 failed all 18 runs on a first-input task; forget bias 2 learned all 12 runs at 20 and 40 steps.

Q: Your LSTM does not learn a long-range task. What do you check first?

A: The gradient reaching the relevant early inputs at initialization. Here it was 3.3e-15 at 100 steps with forget bias 0 and above 1 with forget bias 2.

Interview preparation — theoretical questions

Q: Why does the LSTM's gradient path through the cell state help?

A: Going back one step multiplies the cell-state gradient by the forget gate only, with no weight matrix and no squashing slope. If the forget gates are near 1, the product over many steps stays near 1.

Q: Does that path mean LSTM gradients cannot vanish or explode?

A: No. The direct path vanishes if the forget gates are small. The paths through h and the gates can still shrink or grow: in Section 3 they carried about 47 times more than the direct path.

Q: Why might an LSTM do worse than a plain RNN?

A: With forget gates near 0.5, the cell state halves every step, so the first input's gradient can be smaller than in a clipped RNN. That happened here at 20 steps with forget bias 0.

Q: Why report several seeds?

A: Long-range training is sensitive to initialization and batch order; here the same setting ended at 0.605 for one seed and 0.978 for another. One run can say almost anything.

Key takeaway and what's next

> **KEY TAKEAWAY**
> The LSTM backward pass is checked to within rounding error, through the cell on its own and through the classifier that is trained. In it, the cell-state gradient is multiplied by the forget gate at every step back: exactly Day 63's direct route. Trained on the task the vanilla RNN could not learn at 40 steps, the LSTM learns it in every run when its forget gate starts open (bias 2), and in none when it starts half closed (bias 0). The forget bias is not a detail; on this task it is the difference.

- Two gradient streams: dh through the gates and W, dc through the cell-state line.
- dc_prev = dc * f: the forget gate is the per-step factor on the direct route.
- Gradient-check every output the layer exposes, padding included, then through the real training loss.
- At initialization, forget bias 0 passed 3.3e-15 of the gradient back 100 steps; forget bias 2 passed back more than 1.
- Forget bias 0: 0 of 18 runs learned. Forget bias 2: 12 of 12 at 20 and 40 steps, 4 of 6 at 100.
- Long-range training varies from seed to seed: report several.

Suggested watch order, if starting from zero

- 1. CS231n Winter 2016: Lecture 10: Recurrent Neural Networks, Image Captioning, LSTM (Andrej Karpathy) — the LSTM's gradient flow, explained by the person who taught CS231n's RNN lectures.
- 2. LSTM Backprop Through Time Formula Derivation (Henry Harvin) — the backward-pass formulas derived step by step.

Day 66 preview

- **The GRU, from scratch.** A gated cell with one state instead of two and three gate-like parts instead of four. Built and checked the same way, then set beside the LSTM.
- Today's exercise script, `day65_exercise.py`, is the backward pass on its own, the LSTM version of Day 58's exercise.